



Efficient Memory Management for Large Language Model Serving with PagedAttention

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work

3. 제안 방법론

Main Idea

Contribution

4. 실험 및 결과

Dataset

Baseline

결과

5. 결론 (배운점)

1. Introduction

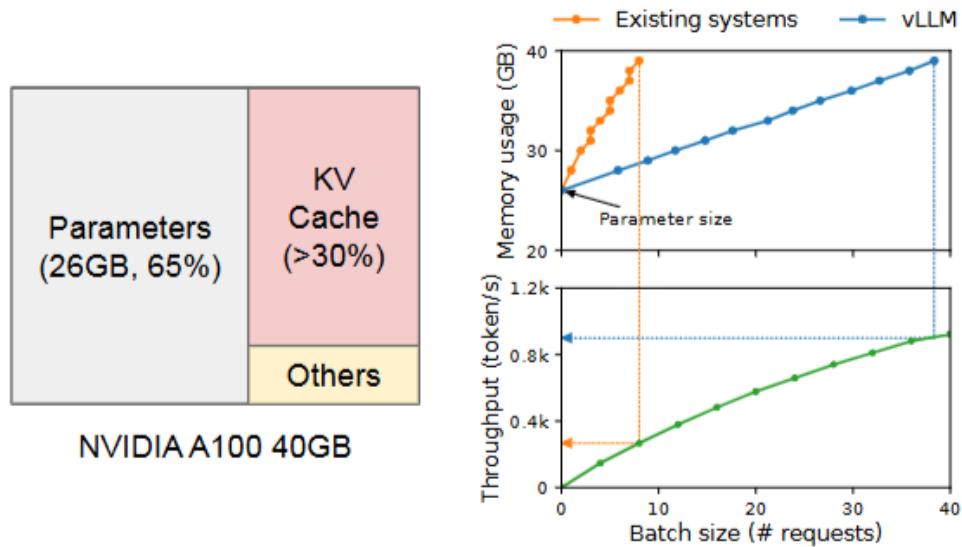
논문이 다루는 분야

- LLM의 등장
 - 비용 비쌈, 많은 수의 하드웨어 가속기 필요
 - LLM 요청 처리 비용은 기존 키워드 쿼리보다 10배 더 비쌈
- LLM 서빙 시스템의 처리량을 늘리는 것(요청당 비용 줄이는 것) 더욱 중요해짐

해당 task에서 기존 연구 한계점

- LLM의 autoregressive 트랜스포머 모델
 - 입력과 이전에 생성된 출력 토큰 시퀀스 기반으로 한 번에 하나의 단어 생성, 종료 토큰 출력할 때까지 반복

- 이 순차적 생성 과정은 워크로드를 memory-bound로 만들어 GPU의 연산능력 충분히 처리하지 못하게 함, 서버 처리량 제한
- 여러 요청을 함께 배치해 처리량 개선 가능
 - 많은 요청을 배치로 처리하려면 각 요청에 대한 메모리 공간의 효율적 관리 필요



40GB RAM을 갖춘 NVIDIA A100 GPU에서 13B-매개변수 LLM의 메모리 분포

- 메모리의 약 65%는 서버 중에 정적으로 유지되는 모델 가중치를 위해 할당
- 30%는 요청의 동적 상태를 저장하는 데 사용
 - 트랜스포머의 경우 어텐션 메커니즘과 관련된 키, 값 텐서로 구성 → KV 캐시
 - 순차적으로 새로운 출력 토큰을 생성하기 위한 이전 토큰의 컨텍스트를 의미
- 나머지 메모리는 LLM 평가 시 생성되는 휘발성 텐서인 활성화 포함한 다른 데이터를 위해 사용
- 모델 가중치는 일정, 활성화는 메모리의 작은 부분만 차지하므로 KV 캐시를 관리하는 방식이 최대 배치 크기의 결정에 중요
- KV 캐시가 비효율적으로 관리될 경우 배치 크기를 크게 제한할 수 있으며, LLM의 처리량에 영향 미침
- 기존 LLM 서버 시스템이 KV 캐시 메모리를 효율적으로 관리하는 데 부족하다는 것 관찰
 - 대부분의 딥러닝 프레임워크가 텐서를 연속적인 메모리 공간에 저장 → 요청의 KV 캐시도 연속적인 메모리 공간에 저장
 - KV 캐시의 특성

- 모델이 새로운 토큰을 생성함에 따라 시간이 지나면 동적으로 증가하고 감소
- 수명과 길이를 미리 알 수 없음

→ 이러한 특성으로 인해 기존의 접근 방식이 비효율적

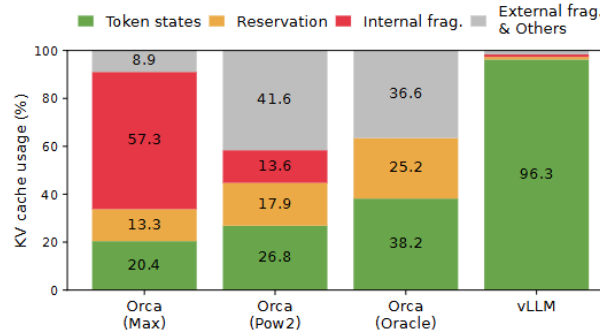


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

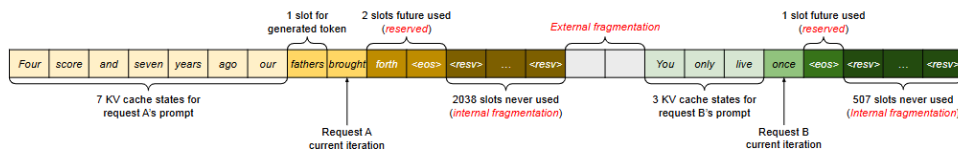


Figure 3. KV cache memory management in existing systems. Three types of memory wastes – reserved, internal fragmentation, and external fragmentation – exist that prevent other requests from fitting into the memory. The token in each memory slot represents its KV cache. Note the same tokens can have different KV cache when at different positions.

1. 내부 및 외부 메모리 단편화로 인한 어려움

- KV 캐시를 연속적인 공간에 할당하기 위해 요청의 최대 길이 가진 연속적인 메모리 청크를 미리 할당
- 요청의 실제 길이가 최대 길이보다 짧을 경우 내부 단편화 초래
- 요청의 전체 청크가 요청의 수명 주기 동안 예약되므로 다른 짧은 요청이 현재 사용되지 않는 청크의 일부 활용 불가
- 사전 할당된 크기가 각 요청마다 다를 수 있으므로 외부 메모리 단편화 발생

2. 메모리 공유 활용 불가

- LLM 서비스는 병렬 샘플링, 빔 서치와 같은 고급 디코딩 알고리즘 사용해 요청당 여러 출력 생성 가능
- 요청은 KV 캐시를 부분적으로 공유할 수 있는 여러 시퀀스로 구성되지만, 시퀀스의 KV 캐시가 별도의 연속적인 공간에 저장되기 때문에 기존 시스템에서는 메모리 공유 불가

논문의 contributions

- **PagedAttention**

- 운영 체제의 메모리 단편화 및 공유 해결책인 가상 메모리와 페이징에서 영감 받은 어텐션 알고리즘
 - 요청의 KV 캐시를 블록으로 분할
 - 각 블록은 고정된 수의 토큰에 대한 어텐션 키와 값을 포함
- KV 캐시를 위한 블록이 연속적인 공간에 저장될 필요 X → OS의 가상 메모리처럼 더 유연한 방식으로 KV 캐시 관리 가능
 - 블록을 페이지로, 토큰을 바이트로, 요청을 프로세스로 생각할 수 있음
 - 상대적으로 작은 블록 사용, 필요에 따라 할당함으로 내부 단편화를 완화, 모든 블록의 크기 동일하므로 외부 단편화 제거
 - 동일한 요청과 관련된 다른 시퀀스 간, 다른 요청 간에서도 메모리 공유 가능

- **vLLM**

- PagedAttention을 기반으로 하는 고처리량 분산 LLM 서빙 엔진
- PagedAttention과 공동 설계된 블록 수준 메모리 관리 및 선점형 요청 스케줄링 사용
- 단일 GPU의 메모리 용량 초과하는 모델 포함하여 GPT, OPT, LLaMA와 같은 인기 LLM을 다양한 크기로 지원
- 평가 결과, 모델 정확도에 영향 미치지 않으면서 sota에 비해 LLM 서빙 처리량을 2-4배 향상
- 더 긴 시퀀스, 더 큰 모델, 더 복잡한 디코딩 알고리즘에서 더 큰 개선

- LLM 서빙에서의 메모리 할당 문제 식별, 서빙 성능에 미치는 영향 정량화
- OS의 가상 메모리 및 페이징에서 영감 받은 PageAttention 제안 — 비연속적인 페이징 메모리에 저장된 KV 캐시에서 작동하는 어텐션 알고리즘
- PageAttention을 기반으로 구축된 분산 LLM 서빙 엔진인 vLLM 설계, 구현
- 다양한 시나리오에서 vLLM 평가, FaterTransformer 및 Orca와 같은 이전 최첨단 솔루션보다 훨씬 뛰어난 성능

2. Related Work

- **일반적인 모델 서빙 시스템 (General Model Serving Systems)**
 - **Clipper, TensorFlow Serving, Nexus, InferLine, Clockwork:** 이러한 초기 시스템들은 단일 또는 다중 모델 서빙을 위한 배치(batching), 캐싱(caching), 배치(placement), 스케줄링(scheduling) 등을 다룸
 - **DVABatch:** 최근 연구로, 다양한 진입점과 출구점을 가진 멀티-엔트리 멀티-엑시트 배치(multi-entry multi-exit batching)를 도입
 - **REEF, Shepherd:** 서빙을 위한 선점(preemption) 메커니즘을 제안
 - **AlpaServe:** 통계적 다중화(statistical multiplexing)를 위해 모델 병렬화(model parallelism)를 활용
 - 이러한 일반 시스템들은 LLM 추론의 자기회귀적 특성(autoregressive property)과 토큰 상태(token state)를 고려하지 않아 최적화 기회를 놓치기 쉽다는 차이점이 있음
- **트랜스포머 특화 서빙 시스템 (Specialized Serving Systems for Transformers)**
 - 트랜스포머 아키텍처의 중요성 때문에 많은 특화 시스템들이 개발됨
 - 이 시스템들은 GPU 커널 최적화, 고급 배치 메커니즘, 모델 병렬화, 파라미터 공유 등을 활용
 - FasterTransformer, Orca와 같은 시스템들이 있음
- **Orca와의 비교**
 - **Orca:** 처리량 최적화에 초점을 맞춘 최신 LLM 서빙 시스템
 - Orca의 반복 수준 스케줄링(iteration-level scheduling)과 vLLM의 PagedAttention은 상호 보완적인 기법
 - 차이점:
 - Orca는 요청을 스케줄링하고 인터리빙하여 더 많은 요청을 병렬 처리함으로써 GPU 활용률과 처리량을 높임
 - vLLM은 메모리 활용률을 높여 더 많은 요청의 작업 집합(working sets)이 메모리에 적합하도록 함으로써 처리량을 높임
 - vLLM은 메모리 단편화를 줄이고 공유를 가능하게 하여 Orca 대비 2-4배의 속도 향상 달성
 - Orca와 같은 세분화된 스케줄링은 메모리 관리를 더 어렵게 만들
- **메모리 최적화 기법 (Memory Optimizations):**

- 컴퓨터와 메모리 용량 간의 격차로 인해 메모리가 학습 및 추론 모두에서 병목 현상이 됨
- 스와핑(Swapping), 재계산(Recomputation): 학습 시 피크 메모리 사용량을 줄이기 위해 활용
- **FlexGen**: LLM 추론에서 제한된 GPU 메모리를 위해 가중치와 토큰 상태를 스와핑하는 방법을 연구, 온라인 서빙 환경 대상은 X
- **OLLA**: 텐서의 생명주기(lifetime)와 위치(location)를 최적화하여 단편화를 줄임, 세분화된 블록 단위 관리나 온라인 서빙을 다루지는 않음
- **FlashAttention**: 어텐션 계산의 피크 메모리를 줄이고 I/O 비용을 절감하기 위해 타일링(tiling) 및 커널 최적화를 적용
- 이 논문은 온라인 서빙 맥락에서 블록 단위 메모리 관리라는 새로운 아이디어를 제시함

3. 제안 방법론

Main Idea

- 이 논문의 핵심 아이디어는 운영체제(OS)의 가상 메모리(virtual memory) 및 페이징(paging) 기법을 대규모 언어 모델(LLM) 서빙 환경의 키-값 캐시(KV cache) 관리에 적용하는 것
- 이를 통해 기존 LLM 서빙 시스템의 고질적인 메모리 단편화(fragmentation) 및 비효율성 문제를 해결, 메모리 사용량을 극적으로 줄여 처리량(throughput)을 향상시킴
- **PagedAttention 알고리즘**

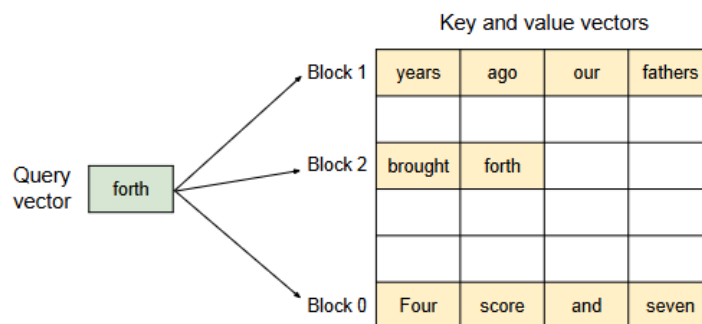


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

- KV 캐시의 페이징(Paging)
 - PagedAttention은 각 LLM 요청의 KV 캐시를 고정된 크기(B)의 KV 블록(KV block)으로 분할
 - OS의 가상 메모리에서 데이터를 페이지(page) 단위로 관리하는 것과 유사
- 비연속적 메모리 할당
 - PagedAttention의 핵심은 KV 블록들이 물리 메모리 상에서 반드시 연속적일 필요가 없다는 것
 - 이는 GPU DRAM 상에서 KV 블록들이 흩어져(non-contiguous) 존재할 수 있음을 의미
 - OS가 논리적 메모리 주소 공간을 물리적 메모리 주소 공간에 유연하게 매핑하는 것과 유사
- 동적 메모리 할당 및 온디맨드(On-demand) 할당
 - vLLM 시스템은 요청 처리 중 새로운 토큰이 생성되어 KV 캐시가 확장될 때, 필요한 만큼만 물리적인 KV 블록을 동적으로 할당
- vLLM 시스템

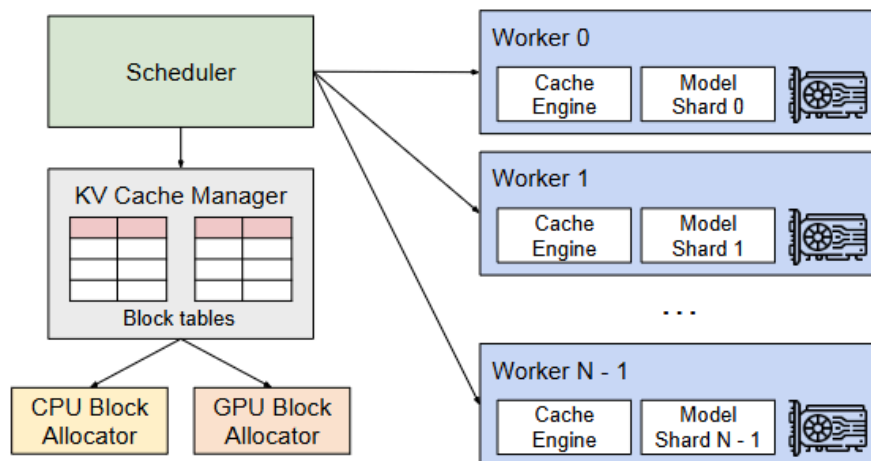


Figure 4. vLLM system overview.

- KV 캐시 매니저
 - vLLM은 PagedAttention을 기반으로 하는 KV Cache Manager를 통해 물리적인 GPU 메모리를 관리

- 이 매니저는 Block Engine을 통해 GPU DRAM을 물리적 KV 블록으로 분할하고 할당
- 블록 테이블 (Block Table)

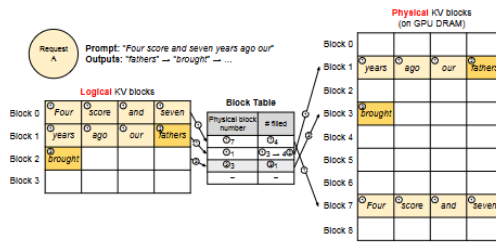


Figure 6. Block table translation in vLLM.

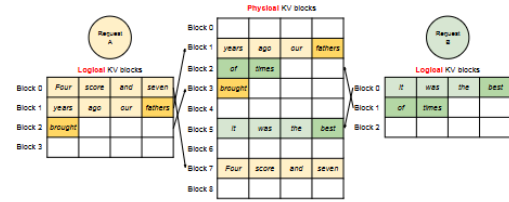


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

- 각 요청의 KV 캐시는 논리적 KV 블록(logical KV blocks)으로 표현되며, KV Cache Manager는 Block Table을 유지하여 각 논리 블록이 어떤 물리적 KV 블록(physical KV block)에 매핑되는지, 그리고 해당 물리 블록에 몇 개의 토큰 데이터가 채워져 있는지(filled positions)를 추적
- 이러한 PagedAttention 기반의 블록 단위 메모리 관리 및 동적 할당 방식을 통해, 기존 시스템에서 발생하던 심각한 내부 단편화(pre-allocation for max length)와 외부 단편화(variable size allocations)를 거의 제거할 수 있음
- 결과적으로 KV 캐시 메모리 낭비를 거의 없애고 더 많은 요청을 GPU 메모리에 적재(batching) 가능
- PagedAttention은 블록 단위로 메모리를 관리하므로, OS의 복사-쓰기(copy-on-write) 메커니즘과 유사하게, 여러 요청 또는 동일 요청 내 여러 시퀀스(예: 빔 서치의 후보들)가 KV 캐시 블록을 공유하는 것을 효율적으로 지원

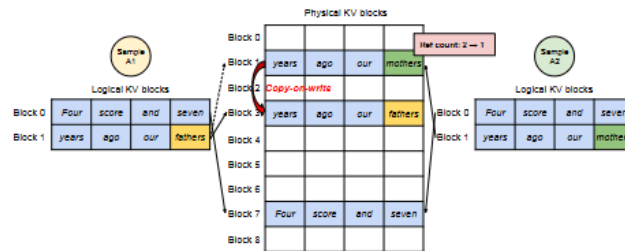


Figure 8. Parallel sampling example.

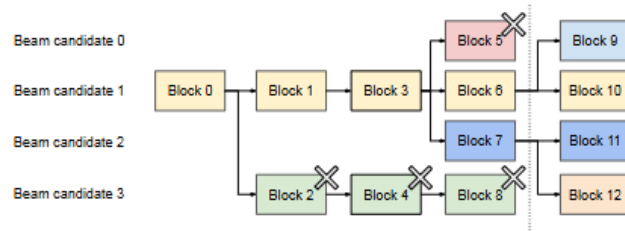


Figure 9. Beam search example.

- 병렬 샘플링, 빔 서치, 공유 접두사(shared prefix)와 같은 복잡한 디코딩 알고리즘에서 메모리 사용량을 크게 절감하는 데 기여

Contribution

- LLM 서빙 시 메모리 할당 문제점을 식별하고 성능 영향을 분석
- 운영체제의 가상 메모리 및 페이징 기법에서 영감받은 PagedAttention 알고리즘을 제안
 - KV 캐시를 비연속적인 페이징 메모리에 저장하는 알고리즘
- PagedAttention을 기반으로 효율적인 메모리 관리를 수행하는 고성능 LLM 서빙 엔진 vLLM을 설계 및 구현
- 다양한 시나리오에서 vLLM의 성능을 평가하여 FasterTransformer 및 Orca와 같은 기존 최신 솔루션 대비 뛰어난 성능을 입증

4. 실험 및 결과

Dataset

- ShareGPT 데이터셋
 - 사용자 공유 대화 데이터로, 긴 입력 프롬프트와 긴 출력 텍스트를 가짐
- Alpaca 데이터셋
 - GPT3.5로 생성된 지시(instruction) 데이터셋, ShareGPT보다 짧은 시퀀스

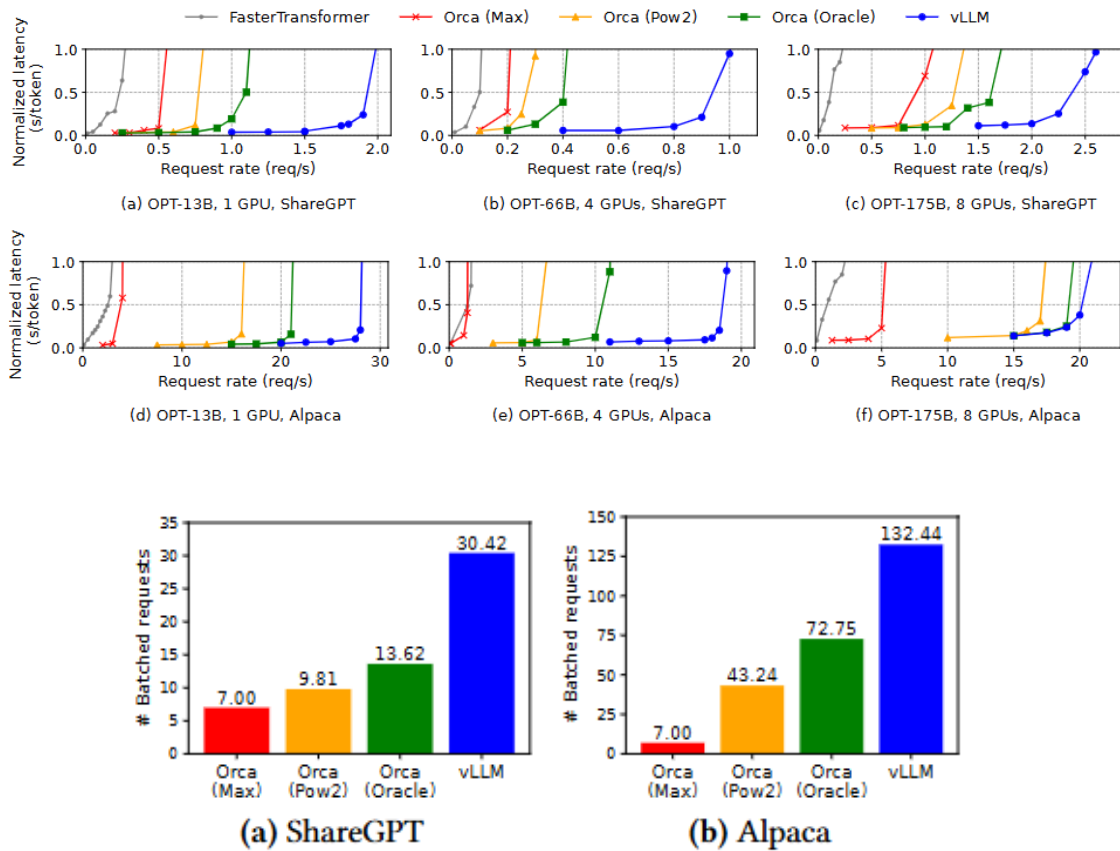
Baseline

- FasterTransformer
 - 지연 시간(latency)에 최적화된 분산 추론 엔진
 - 동적 배치 메커니즘을 사용하지만, fine-grained 스케줄링의 부족
- Orca

- 처리량(throughput)에 최적화된 LLM 서빙 시스템
- Orca (Oracle): 실제 생성될 출력 길이를 미리 아는 경우
- Orca (Pow2): 출력 길이를 최대 2배까지 초과 할당
- Orca (Max): 모델의 최대 시퀀스 길이(2048 토큰)까지 공간을 할당

결과

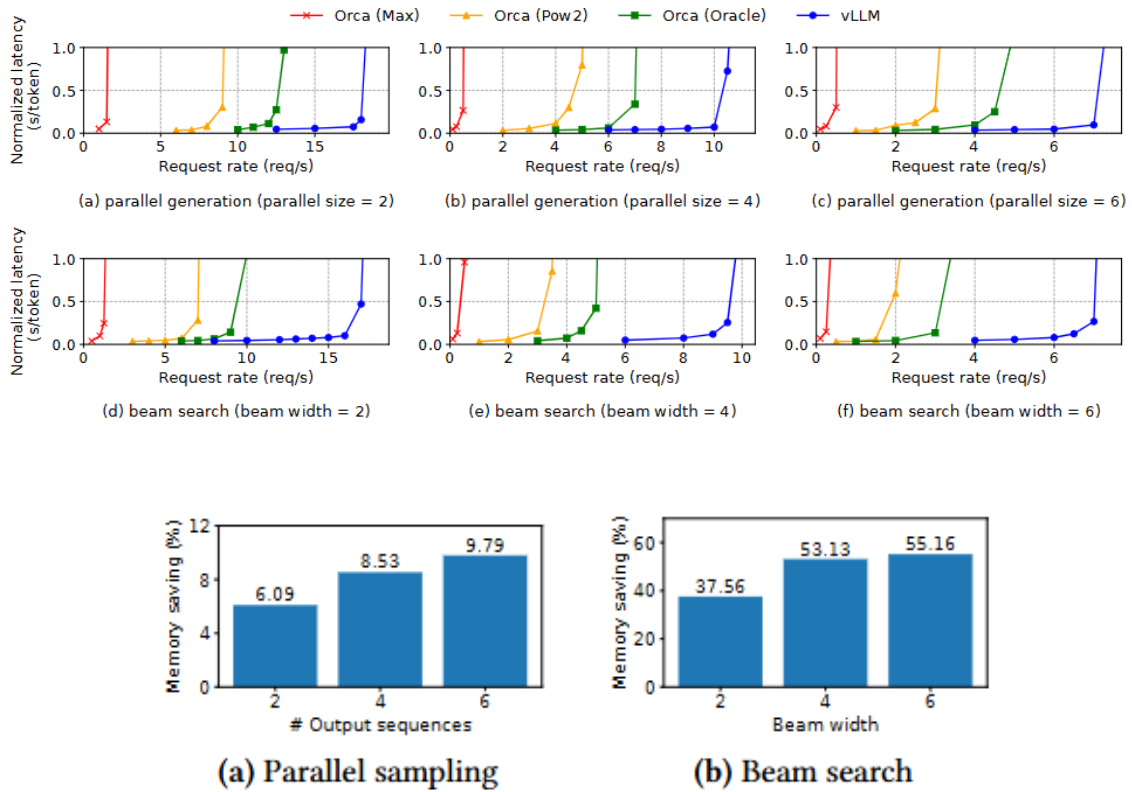
• 기본 샘플링 (Basic Sampling)



- ShareGPT 및 Alpaca 데이터셋에서 vLLM은 Orca (Oracle) 대비 1.7-2.7배, Orca (Max) 대비 2.7-8배 높은 요청 처리율을 유지하면서 유사한 지연 시간을 보임
- vLLM은 PagedAttention을 통해 더 많은 요청을 배치할 수 있어 더 높은 처리량 달성
- OPT-13B 모델에서 vLLM은 Orca (Oracle) 대비 2.2배, Orca (Max) 대비 4.3배 더 많은 요청을 동시에 처리
- FasterTransformer 대비 최대 22배 높은 요청 처리율을 보임

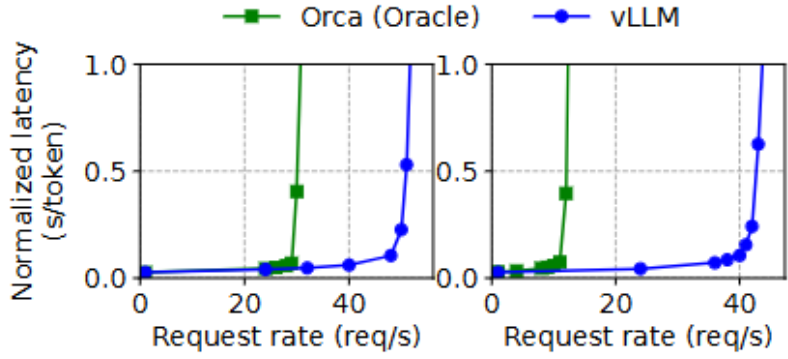
- FasterTransformer가 fine-grained 스케줄링 없이 메모리를 비효율적으로 관리하기 때문

• 병렬 샘플링 및 빔 서치



- 더 복잡한 디코딩 알고리즘일수록 vLLM의 메모리 공유 장점이 두드러짐
- 빔 서치(beam width=6)에서 vLLM은 Orca (Oracle) 대비 1.3배(기본 샘플링)에서 2.3배로 성능 향상 폭이 커짐
- 메모리 절감 효과: 병렬 샘플링에서 6.1%-9.8% (ShareGPT: 16.2%-30.5%), 빔 서치에서 37.6%-55.2% (ShareGPT: 44.3%-66.3%)

• 공유 접두사 (Shared prefix)



(a) 1-shot prefix prompt (b) 5-shot prefix prompt

- 입력 프롬프트가 긴 예시(few-shot)를 공유할수록 vLLM의 성능 향상 폭이 커짐
 - 1-shot 공유 시 1.67배, 5-shot 공유 시 3.58배의 처리량 향상

• 챗봇

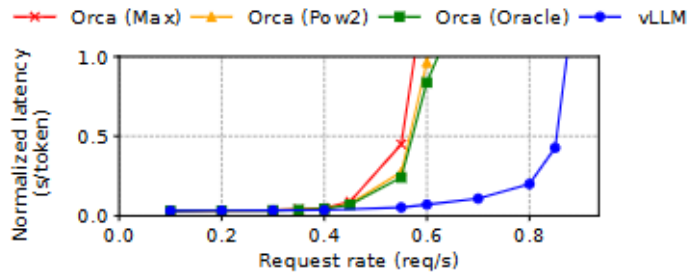
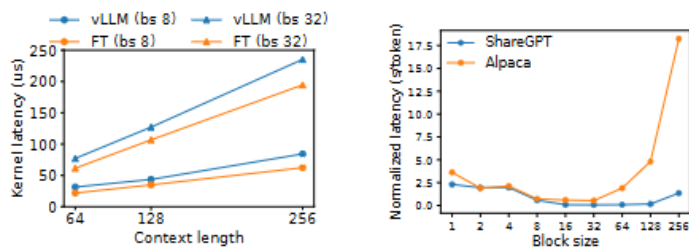


Figure 17. Performance on chatbot workload.

- 챗봇 워크로드에서 vLLM은 긴 대화 기록으로 인해 발생하는 긴 프롬프트 처리에서 Orca 기반 시스템 대비 2배 높은 요청 처리율을 보임

• 커널 최적화



(a) Latency of attention kernels. (b) End-to-end latency with different block sizes.

Figure 18. Ablation experiments.

- PagedAttention의 오버헤드(블록 테이블 접근, 분기 처리 등)로 인해 어텐션 커널 자체의 지연 시간이 FasterTransformer 대비 20-26% 증가
- 하지만 이는 전체 모델 연산 중 어텐션 부분에만 영향을 미치며, vLLM의 전체적인 엔드-투-엔드 성능은 오히려 향상되는 결과

• 블록 크기 영향

- 블록 크기가 너무 작으면 GPU 병렬성 활용이 저하되고, 너무 크면 내부 단편화가 증가하고 공유 가능성이 줄어듦
- ShareGPT에서는 16-128, Alpaca에서는 16-32에서 좋은 성능

• 재계산(Recomputation) vs. 스와핑(Swapping)

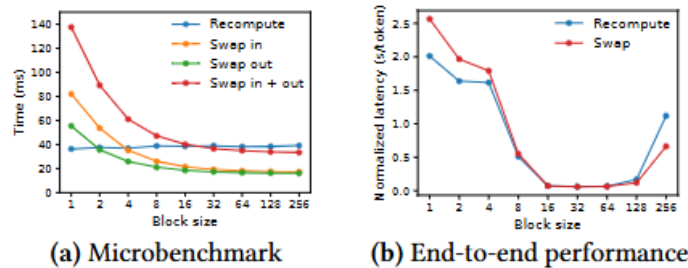


Figure 19. (a) Overhead of recomputation and swapping for different block sizes. (b) Performance when serving OPT-13B with the ShareGPT traces at the same request rate.

- 스와핑은 작은 블록 크기에서 CPU-GPU 간 작은 데이터 전송이 빈번하여 PCIe 대역폭 병목으로 인해 큰 오버헤드를 유발
- 재계산은 블록 크기에 관계없이 일정한 오버헤드를 가지며, 전반적으로 스와핑보다 효율적이거나 유사한 성능을 보임

5. 결론 (배운점)

• 연구의 의의 및 한계점

- 의의:
 - LLM 서빙의 주요 병목 지점인 KV 캐시 메모리 관리에 대한 해결책 제시
 - 운영체제의 가상 메모리 및 페이징 기법을 LLM 서빙에 적용하여 메모리 단편화를 극복, 다양한 디코딩 시나리오에서 유연한 공유를 가능하게 함
 - 기존 시스템 대비 2-4배의 처리량 향상을 달성

- PagedAttention과 vLLM은 LLM 서빙 시스템의 메모리 효율성을 크게 개선하여, 더 많은 요청을 동시에 처리할 수 있게 함

◦ **한계점:**

- PagedAttention 커널은 기존 시스템 대비 약간의 오버헤드를 가짐 (20-26% 더 높은 어텐션 커널 지연 시간).
- 전체 성능에는 큰 영향이 없었지만, 다른 GPU 워크로드에 적용 시 이러한 간접 참조(memory indirection) 오버헤드가 성능 저하를 유발할 수 있음
- GPU 연산 성능의 지속적인 향상과 메모리 용량 증가율의 격차는 여전히 LLM 서빙의 근본적인 과제로 남아있음



- 운영체제의 핵심 개념(가상 메모리, 페이징, 복사-쓰기)을 LLM 서빙이라는 새로운 분야에 창의적으로 적용한 점이 인상 깊었다.
- 병렬 샘플링, 빔 서치, 공유 접두사 등 다양한 복잡한 디코딩 알고리즘에서의 효율성을 보여준 것이 좋았다.



ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

1. Introduction

논문이 다루는 분야

- 확률적 그래디언트 최적화(stochastic gradient-based optimization)
 - 과학 및 공학의 많은 분야에서 중요
 - 매개변수에 대한 최대화 또는 최소화를 요구하는 스칼라 매개변수화된 목적 함수의 최적화로 구성된 문제
 - 함수가 매개변수에 대해 미분 가능한 경우, 그래디언트 하강법으로 효율적인 최적화
 - 모든 매개변수에 대한 1차 편미분 계산은 함수 자체를 평가하는 것과 동일한 계산 복잡성 가지기 때문

해당 task에서 기존 연구 한계점

- 목적 함수는 종종 확률적
 - 많은 목적 함수는 다른 데이터 부분 샘플에서 평가된 하위 함수의 합으로 구성됨
 - 이 경우 개별 하위 함수에 대한 그래디언트 단계를 취함으로써 더 효율적으로 최적화
 - 확률적 그래디언트 하강법(SGD)
 - 목적 함수는 데이터 부분 샘플링 외에 다른 노이즈 소스 가질 수도 있음(dropout 등)
 - 이런 모든 노이즈가 있는 목적 함수에 대해 효율적인 확률적 최적화 기법 필요

- 논문은 고차원 매개변수 공간을 가진 확률적 목적 함수의 최적화에 초점

논문의 contributions

- **Adam**
 - 적은 메모리 요구량으로 1차 그래디언트만을 필요로 함
 - 그래디언트의 1차 및 2차 모멘트 추정치로부터 각 매개변수에 대한 개별 적응적 학습률을 계산
 - 적응적 모멘트 추정(adaptive moment estimation) → Adam
 - AdaGrad와 RMSProp의 장점을 결합
 - 희소한 그래디언트에서 잘 작동, 온라인 및 가변적인 설정에서 잘 작동
- Adam의 장점
 - 매개변수 업데이트의 크기가 그래디언트의 재스케일링에도 변하지 않음
 - 스텝 크기가 스텝 크기 하이퍼파라미터에 의해 대략적으로 제한됨
 - 희소 그래디언트와 작동, 일종의 스텝 크기 어닐링(annealing, 점진적 감소)을 자연스럽게 수행
- 다양한 모델과 데이터셋에 대해 일관되게 우수한 성능 보임
- Adam이 대규모 고차원 기계 학습 문제에 확장 가능한 다목적 알고리즘이라는 결과 도출